SIEM Lab — Snort 部分实验报告

项目仓库根目录：`/Users/liangyibo/Desktop/SIEM`
目标：在 Docker 化的 Wazuh SIEM 实验环境中，基于 Snort 实现网络层多阶段攻击检测，并在 Wazuh 端进行关联、升级到 HIGH / CRITICAL 级别告警。

目录

1. 实验目标与总体架构
2. 攻击场景与杀伤链（Kill-Chain）设计
3. Snort 规则链（SIDs 1010101–1010105）
4. Wazuh 侧规则：阶段标记与跨阶段关联
5. 修改清单（文件与组件）
6. 端到端运行流程（实验步骤）
7. 在 Wazuh Dashboard 中的验证查询
8. 问题与排错记录

1. 实验目标与总体架构

实验在一个单主机 Docker 环境中复现完整的 SIEM 处理链条：**攻击流量 → Snort 网络层检测 → Wazuh 代理采集 → Wazuh 管理端规则引擎与关联 → 在 Wazuh Dashboard 中可视化**。集群分为三个逻辑节点：

- **siem-server**：容器 `wazuh.manager`、`wazuh.indexer`、`wazuh.dashboard`（镜像版本 4.14.3）。
- **victim**：容器内并行运行 `Wazuh agent`、`Snort`（嗅探 `eth0`）与一个最小化的 HTTP 服务（Python `http.server`，端口 8080）。
- **attacker**：容器，预装 `ping` / `nmap` / `curl`，用于生成攻击流量。

三者位于同一 Docker bridge 网络 `siem-lab-net`，因此 `attacker → victim` 的流量会被 victim 容器内 Snort 嗅探到，alert 写入 `/var/log/snort/alert`；Wazuh agent 以 `snort-full` 格式读取后经 `1514/tcp` 上送到 manager。

2. 攻击场景与杀伤链设计

为了让规则链本身**具有逻辑**（而非罗列互不相关的探测），场景按 MITRE ATT&CK 的阶段组织成一条外部攻击者针对 Web 服务的杀伤链：

阶段	MITRE 阶段	攻击流量	Snort SID	Wazuh tag 规则
1	Reconnaissance (TA0043)	`ping -c 3 victim`	`1010101`	`100360` (level 5)
2	Discovery / Active Scanning (T1046)	`nmap -sS -p 22,80,445,3306,8080`	`1010102`	`100361` (level 6)
3	Initial Access — Web probe (T1190)	`curl /admin` 等敏感路径	`1010103`	`100362` (level 8)
4	Exploitation — SQLi / LFI	`?q=' OR 1=1`、`?path=../../etc/passwd`	`1010104`	`100363` (level 10)

阶段	MITRE 阶段	攻击流量	Snort SID	Wazuh tag 规则
5	Exfiltration / C2	`POST` + UA `python-requests`	`1010105`	`100364` (level 11)

在 Wazuh 端再做一次**跨阶段关联**（同源 IP，窗口 600 s）：

Wazuh 规则	条件	级别	Dashboard 严重度
`100390`	≥ 3 个 `snort_lab_chain` 事件	12	HIGH
`100391`	≥ 5 个 `snort_lab_chain` 事件（完整链）	15	CRITICAL

3. Snort 规则链（`docker/victim/snort-lab-chain.rules`）

该文件是本实验自写的规则集合（五条规则，SID `1010101` — `1010105`， `rev:11`）。HTTP 相关的三条规则使用 Snort 2.9 的 `http_inspect` 预处理器修饰符（`http_uri` / `http_method` / `/H`），以便在 `HTTP_PORTS`（含 8080）上对规范化后的 URI / 方法 / 头部进行匹配。

```
# ==============================================================================
# SIEM lab — Snort rule chain coherente (kill-chain externo contra victim)
# SIDs 1010101..1010105 (una etapa cada uno). rev:11
# Narrativa:
#   1. Recon      -> ping sweep            (ICMP echo)
#   2. Scan       -> multi-port SYN        (22,80,445,3306,8080)
#   3. Web probe  -> rutas sensibles       (/admin, /login, /.env, ...)
#   4. Exploit    -> payload SQLi / LFI    (UNION SELECT, 1=1, ../, /etc/passwd)
#   5. Exfil/C2   -> POST con UA de automatización (python-requests, sqlmap...)
# ==============================================================================

alert icmp any any -> $HOME_NET any (msg:"LAB Stage1 Reconnaissance: ICMP echo probe";
  itype:8; icode:0; sid:1010101; rev:11; classtype:misc-activity; priority:3;)

alert tcp any any -> $HOME_NET [22,80,445,3306,8080]
  (msg:"LAB Stage2 Service Discovery: SYN probe on well-known port";
   flags:S,12; sid:1010102; rev:11; classtype:attempted-recon; priority:3;)

alert tcp any any -> $HOME_NET $HTTP_PORTS
  (msg:"LAB Stage3 Initial Access: HTTP request to sensitive path";
   flow:to_server,established; content:"/"; http_uri; depth:1;
   pcre:"/^\/(admin|login|phpmyadmin|wp-admin|\.env|\.git|config)/Ui";
   sid:1010103; rev:11; classtype:web-application-attack; priority:2;)

alert tcp any any -> $HOME_NET $HTTP_PORTS
  (msg:"LAB Stage4 Exploit attempt: SQLi or path-traversal payload in HTTP URI";
   flow:to_server,established;
   pcre:"/(union[\s%+]+select|1=1|\.\.\/|\/etc\/passwd|%27|--[\s%+])/Ui";
   sid:1010104; rev:11; classtype:web-application-attack; priority:1;)

alert tcp any any -> $HOME_NET $HTTP_PORTS
  (msg:"LAB Stage5 Exfiltration/C2: HTTP POST with automation User-Agent";
   flow:to_server,established; content:"POST"; http_method;
   pcre:"/(python-requests|sqlmap|curl\/|wget\/|Go-http-client|libwww-perl)/Hi";
   sid:1010105; rev:11; classtype:trojan-activity; priority:1;)
```

> **关于 Docker bridge 下的校验和偏移**：Docker bridge 网络常把 TCP checksum 卸载给虚拟网卡，数据包到达 Snort 时 checksum 字段为 0，Snort 默认会丢弃这些包；只有 `nmap -sS` 这种自行计算 checksum 的 SYN 包能被看见。解决办法

> 是启动 Snort 时加 `-k none` 关闭校验和验证（见下一节的 entrypoint）。

4. Wazuh 侧规则：阶段标记与跨阶段关联（`config/wazuh_manager/rules/snort_correlation_rules.xml`）

Wazuh 的 Snort 解码器会把所有 Snort 告警统一标记成 `rule.id: 20101`。本实验在其之上再做两层规则：

- **阶段标签规则** `100360` – `100364`：用 `<if_sid>20101</if_sid>` 作为基规则，通过 `<match>1:1010101:11</match>` 等字符串在 `full_log` 中识别 SID，从而把每个阶段打上公共组 `snort_lab_chain`；`ignore="60"` 做 60 秒去抖，避免同一阶段因为多包而把频次撑爆。
- **关联规则** `100390` / `100391`：`<if_matched_group>snort_lab_chain</if_matched_group>` + `<same_source_ip />` + `frequency` + `timeframe="600"`，把同源 IP 在 10 分钟内多阶段的事件聚合为 HIGH / CRITICAL。

```
<!--
  Correlación SIEM lab sobre alertas Snort (rule.id=20101 del decoder snort).
  - 100360..100364: etiqueta por etapa (dedup 60s por regla)
  - 100390: mismo origen, >=3 eventos etiqueta snort_lab_chain / 600s -> HIGH
  - 100391: mismo origen, >=5 eventos etiqueta snort_lab_chain / 600s -> CRITICAL
  SIDs Snort referenciados: 1010101..1010105 rev:11
-->
<group name="snort,ids,lab,">

  <rule id="100360" level="5" ignore="60">
    <if_sid>20101</if_sid>
    <match>1:1010101:11</match>
    <description>LAB kill-chain [1/5] Reconnaissance: ICMP echo probe detected</description>
    <group>snort_lab_chain,attack.recon,</group>
  </rule>

  <rule id="100361" level="6" ignore="60">
    <if_sid>20101</if_sid>
    <match>1:1010102:11</match>
    <description>LAB kill-chain [2/5] Service Discovery: SYN probe on well-known port</description>
    <group>snort_lab_chain,attack.scan,</group>
  </rule>

  <rule id="100362" level="8" ignore="60">
    <if_sid>20101</if_sid>
    <match>1:1010103:11</match>
    <description>LAB kill-chain [3/5] Initial Access: HTTP request to sensitive path</description>
    <group>snort_lab_chain,attack.web_probe,</group>
  </rule>

  <rule id="100363" level="10" ignore="60">
    <if_sid>20101</if_sid>
    <match>1:1010104:11</match>
    <description>LAB kill-chain [4/5] Exploit: SQLi or path-traversal payload in HTTP request</descriptio
    <group>snort_lab_chain,attack.exploit,</group>
  </rule>

  <rule id="100364" level="11" ignore="60">
    <if_sid>20101</if_sid>
    <match>1:1010105:11</match>
    <description>LAB kill-chain [5/5] Exfiltration/C2: HTTP POST with automation User-Agent</description>
    <group>snort_lab_chain,attack.exfil,</group>
  </rule>

  <rule id="100390" level="12" frequency="3" timeframe="600">
    <if_matched_group>snort_lab_chain</if_matched_group>
    <same_source_ip />
    <description>LAB HIGH severity: multi-stage Snort kill-chain correlated (same source, >=3 stages)</des
    <group>snort_lab_correlation,attack.killchain,</group>
  </rule>

  <rule id="100391" level="15" frequency="5" timeframe="600">
    <if_matched_group>snort_lab_chain</if_matched_group>
    <same_source_ip />
    <description>LAB CRITICAL severity: full Snort kill-chain correlated from single source (>=5 stages)</
    <group>snort_lab_correlation,attack.killchain,</group>
  </rule>

</group>
```

该文件通过 `docker-compose.yml` 中 `./config/wazuh_manager/rules → /var/ossec/etc/rules` 的挂载加入 manager 容器，和官方规则一同加载。

5. 修改清单（文件与组件）

文件 / 组件	类型	修改要点
`docker/victim/snort-lab-chain.rules`	新增	Snort 五阶段规则链（SID `1010101` – `1010105`，`rev:11`），HTTP 规则使用 `http_inspect` 修饰符。
`docker/victim/victim-entrypoint.sh`	修改	① 若 `local.rules` 还没有 `sid:1010101`，把 `lab-chain.rules` 追加进去；② Snort 启动参数增加 `-k none` 关闭校验和验证；③ 用 Python 片段去重 `ossec.conf` 中的 `snort-alerts` `localfile` 块，并以 `snort-full` 格式采集 `/var/log/snort/alert`。
`docker/victim/Dockerfile`	修改	在构建时把规则链复制到镜像内的 `/etc/snort/rules/lab-chain.rules`。
`config/wazuh_manager/rules/snort_correlation_rules.xml`	新增	阶段标签规则 `100360` – `100364` + 关联规则 `100390`（HIGH）/ `100391`（CRITICAL）。
`scripts/run-snort-chain-scenario.sh`	新增	攻击编排脚本，在 `attacker` 容器中依序生成五个阶段的网络流量。
`docker-compose.yml`	既有（复用）	通过 `./config/wazuh_manager/rules → /var/ossec/etc/rules` 与 `./config/wazuh_manager/decoders → /var/ossec/etc/decoders` 的挂载，将自定义规则并入 manager；`victim` 具备 `cap_add: NET_ADMIN / NET_RAW` 以便 Snort 抓包。
`README.md`	修改	增加“Escenario Snort: kill-chain externo contra victim”章节，表格对应阶段—SID—tag 规则；给出 Dashboard 查询样例。

5.1 `victim-entrypoint.sh` 相关片段

```
# Keep exactly one snort localfile block and parse fast alerts with snort-full.
python3 - <<'PY'
from pathlib import Path
import re
path = Path("/var/ossec/etc/ossec.conf")
text = path.read_text()
pattern = re.compile(
    r"<localfile>\s*<log_format>[^<]+</log_format>"
    r"\s*<location>/var/log/snort/alert</location>"
    r"\s*<alias>snort-alerts</alias>\s*</localfile>\s*",
    re.MULTILINE,
)
text = pattern.sub("", text)
path.write_text(text)
PY
append_localfile "/var/log/snort/alert" "snort-alerts" "snort-full"

# ... (agent enrollment, wazuh-control start) ...

# Merge lab rules into local.rules (once per fresh container).
if [ -f /etc/snort/rules/local.rules ] \
   && [ -f /etc/snort/rules/lab-chain.rules ] \
   && ! grep -q "sid:1010101" /etc/snort/rules/local.rules; then
  cat /etc/snort/rules/lab-chain.rules >> /etc/snort/rules/local.rules
fi

# -k none: skip checksum validation. Docker bridges deliver packets with
# TCP checksum offload delegated to hardware -> Snort would drop them otherwise
# and real HTTP traffic (curl) would not trigger any rule beyond the SYN phase.
snort -i "${MONITOR_INTERFACE}" -A fast -k none -q \
      -c /etc/snort/snort.conf -l /var/log/snort \
      >/var/log/snort/snort.stdout.log 2>/var/log/snort/snort.stderr.log &
```

5.2 攻击编排脚本 `scripts/run-snort-chain-scenario.sh`

```bash
#!/usr/bin/env bash
set -euo pipefail
TARGET_HOST="${1:-victim}"
TARGET_PORT="${2:-8080}"
banner() { printf '\n==========================\n%s\n==========================\n' "$*"; }

banner "Stage 1 (Recon) — ping sweep"
docker compose exec attacker ping -c 3 "${TARGET_HOST}" || true

banner "Stage 2 (Service discovery) — nmap multi-puerto"
docker compose exec attacker nmap -sS \
  -p "22,80,445,3306,${TARGET_PORT}" --max-retries 1 -T4 "${TARGET_HOST}" || true

banner "Stage 3 (Web probe) — rutas sensibles"
for p in /admin /login /.env /phpmyadmin /wp-admin /config; do
  docker compose exec attacker curl -s -o /dev/null \
    "http://${TARGET_HOST}:${TARGET_PORT}${p}" || true
done

banner "Stage 4 (Exploit) — SQLi / path-traversal"
docker compose exec attacker curl -s -o /dev/null \
  "http://${TARGET_HOST}:${TARGET_PORT}/search?q=%27+OR+1=1+--+" || true
docker compose exec attacker curl -s -o /dev/null \
  "http://${TARGET_HOST}:${TARGET_PORT}/index?id=1+UNION+SELECT+1,2,3" || true
docker compose exec attacker curl -s -o /dev/null \
  "http://${TARGET_HOST}:${TARGET_PORT}/file?path=../../../etc/passwd" || true

banner "Stage 5 (Exfil/C2) — POST con UA automatizada"
for i in 1 2 3; do
  docker compose exec attacker curl -s -o /dev/null \
    -X POST -A "python-requests/2.28.1" \
    -d "exfil=$(head -c 256 /dev/urandom | base64 | tr -d '\n')" \
    "http://${TARGET_HOST}:${TARGET_PORT}/upload" || true
done
```

6. 端到端运行流程

一次完整的事件在系统里的流转路径：

```
attacker(容器)
  │ ping/nmap/curl 流量
  ▼
eth0 (bridge 网卡, siem-lab-net)
  │
  ▼
victim(容器) ── Snort 嗅探 (-k none, -A fast) ──► /var/log/snort/alert
                                                          │
                                                          ▼
                                                Wazuh agent (snort-full)
                                                          │  1514/tcp
                                                          ▼
                                                wazuh.manager
                                                  │   └─ decoder "snort"     → rule.id: 20101
                                                  │   └─ 100360..100364     → tag group: snort_lab_chain
                                                  │   └─ 100390/100391      → 同源 IP 关联 (HIGH/CRITICAL)
                                                  ▼
                                                wazuh.indexer (OpenSearch)
                                                  │
                                                  ▼
                                                wazuh.dashboard (浏览器)
```

6.1 一次跑通实验的命令（宿主机 shell）

```
# ① 第一次部署：生成 indexer 证书（已生成可跳过）
docker compose -f generate-indexer-certs.yml run --rm generator

# ② 构建并启动全部容器
docker compose up -d --build

# ③ 确认五个容器都在运行
docker compose ps

# ④ 执行多阶段攻击脚本
./scripts/run-snort-chain-scenario.sh

# ⑤ 直接在 victim 上看 Snort 原始告警
docker compose exec victim tail -n 120 /var/log/snort/alert

# ⑥ 在 manager 上确认 Wazuh 关联规则已触发
docker compose exec wazuh.manager bash -lc \
  'tail -n 400 /var/ossec/logs/alerts/alerts.log | \
   grep -E "Rule: (10036[0-4]|10039[01])" | tail -n 40'
```

7. Wazuh Dashboard 验证查询

浏览器打开 `https://localhost:5602`（用户 `admin` / 密码 `SecretPassword`），进入 Discover，索引选 `wazuh-alerts-*`：

① 五阶段 Snort 原始命中（每个数据包一行）：

```
agent.name:"victim-agent"
  AND location:"/var/log/snort/alert"
  AND data.id:(
    "1:1010101:11" OR "1:1010102:11" OR "1:1010103:11"
    OR "1:1010104:11" OR "1:1010105:11"
  )
```

② 每阶段已被 Wazuh 打上标签（去抖 60 s，便于叙事）：

```
agent.name:"victim-agent"
  AND rule.id:(100360 OR 100361 OR 100362 OR 100363 OR 100364)
```

③ 最终关联告警（HIGH / CRITICAL）：

```
agent.name:"victim-agent" AND rule.id:(100390 OR 100391)
```

实际验证时，`100391`（level 15，CRITICAL）会带上 `Src IP: 172.20.0.6`（attacker）和 `Dst IP: 172.20.0.5:8080`（victim），并在正文里列出对应阶段的 Snort 原始 `full_log`，构成完整的"多步骤攻击"叙事。

8. 问题与排错记录

问题	症状	解决
Docker bridge 下 TCP checksum offload	只看到 `nmap -sS` 的 SYN 告警；`curl` 产生的 HTTP 请求在 Snort 里完全没有命中。	Snort 启动参数加 `-k none`，关闭校验和验证。
Wazuh 自定义规则一直不触发	只能看到默认的 `rule.id: 20101`；曾尝试 `<field name="id">`（报 "field is static"）或匹配 `full_log` 中 `LAB Stage` 字符串均不稳定。	改为 `<if_sid>20101</if_sid>` 作为基规则，并用 `<match>1:101010X:11</match>` 精确匹配 SID 字符串。
Wazuh agent 采集 Snort 重复	`wazuh-logcollector` 多次重复采集 `/var/log/snort/alert`。	在 `victim-entrypoint.sh` 中先用 Python 正则从 `ossec.conf` 里删除已有的 `snort-alerts` `localfile` 块，再统一用 `snort-full` 格式追加一条。
HTTP 规则用 `content:"GET "` 不命中	Snort 在 `HTTP_PORTS` 上启用 `http_inspect` 预处理器，内容匹配走的是规范化后的缓冲区。	规则改用 `content:"/"; http_uri; depth:1;` 和 `pcre:"/.../Ui"`（URI 缓冲区）、`content:"POST"; http_method;` 和 `pcre:"/.../Hi"`（HTTP 头）。

报告生成：根据仓库当前状态（`rev:11`）整理。
关键路径索引：
— Snort 规则链：`docker/victim/snort-lab-chain.rules`
— Snort 启动与日志采集：`docker/victim/victim-entrypoint.sh`
— Wazuh 关联规则：`config/wazuh_manager/rules/snort_correlation_rules.xml`
— 攻击编排脚本：`scripts/run-snort-chain-scenario.sh`